



دانشگاه صنعتی شریف
دانشکده‌ی مهندسی کامپیوتر

نظریه‌ی زبان‌ها و ماشین‌ها

نمونه‌سوال‌ها

پاسخ‌نامه‌ی مجموعه‌ی ۶

عبارت‌های منظم و ماشین‌های متناهی، زبان‌های نامنظم

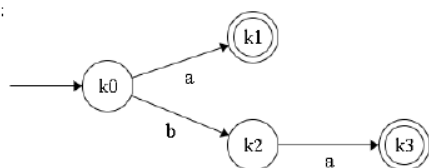
نیم‌سال اول - سال تحصیلی ۰۳-۰۴

۱ عبارت‌های منظم

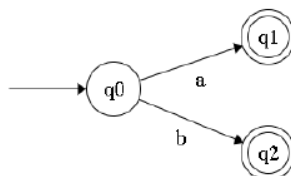
۱-۱

(a

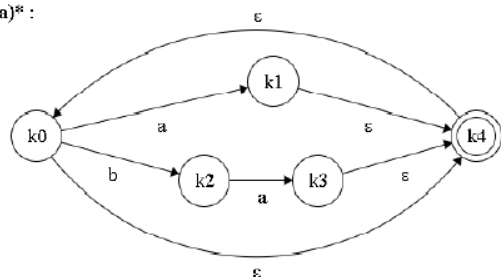
$a \cup ba :$



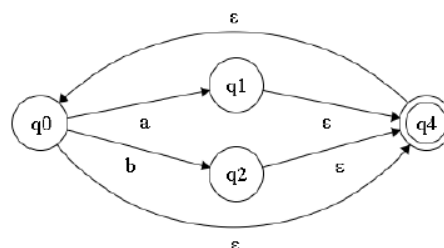
$a \cup b :$



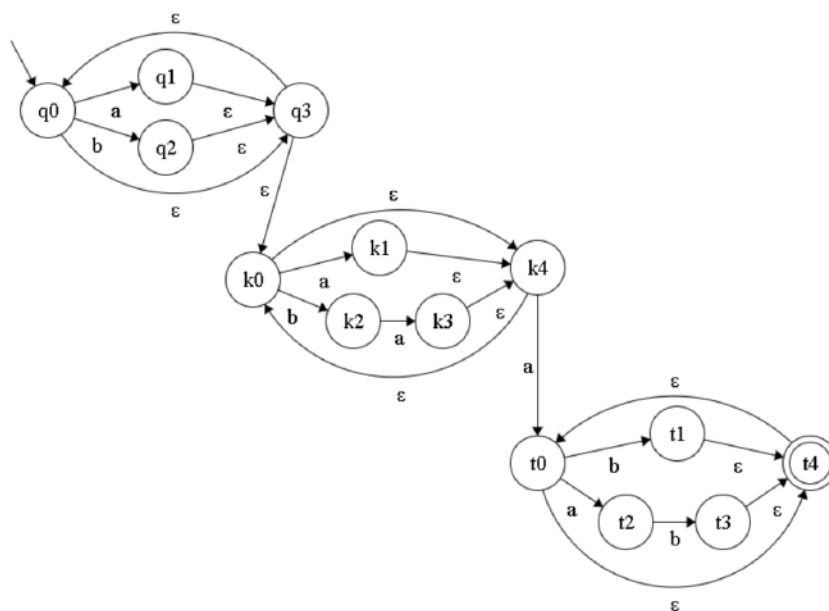
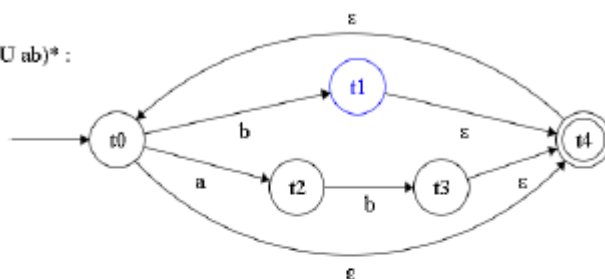
$(a \cup ba)^* :$



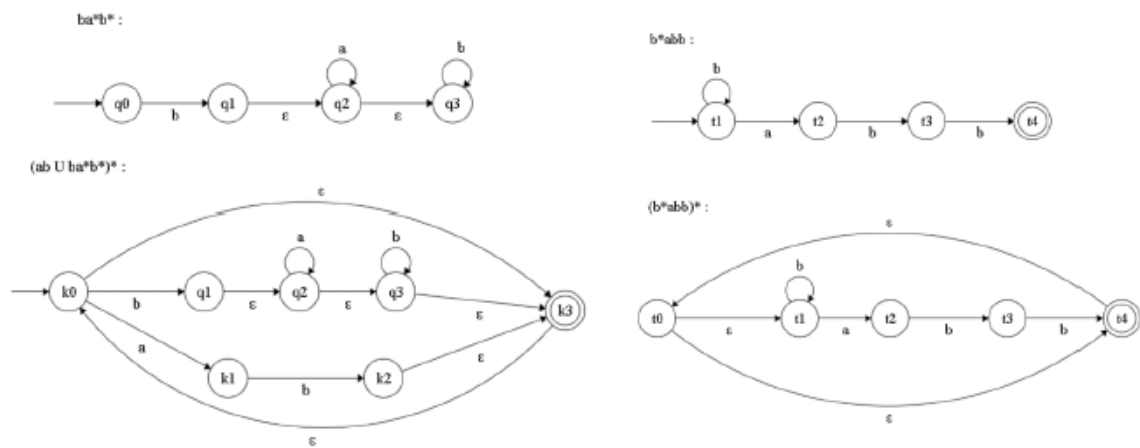
$(a \cup b)^* :$



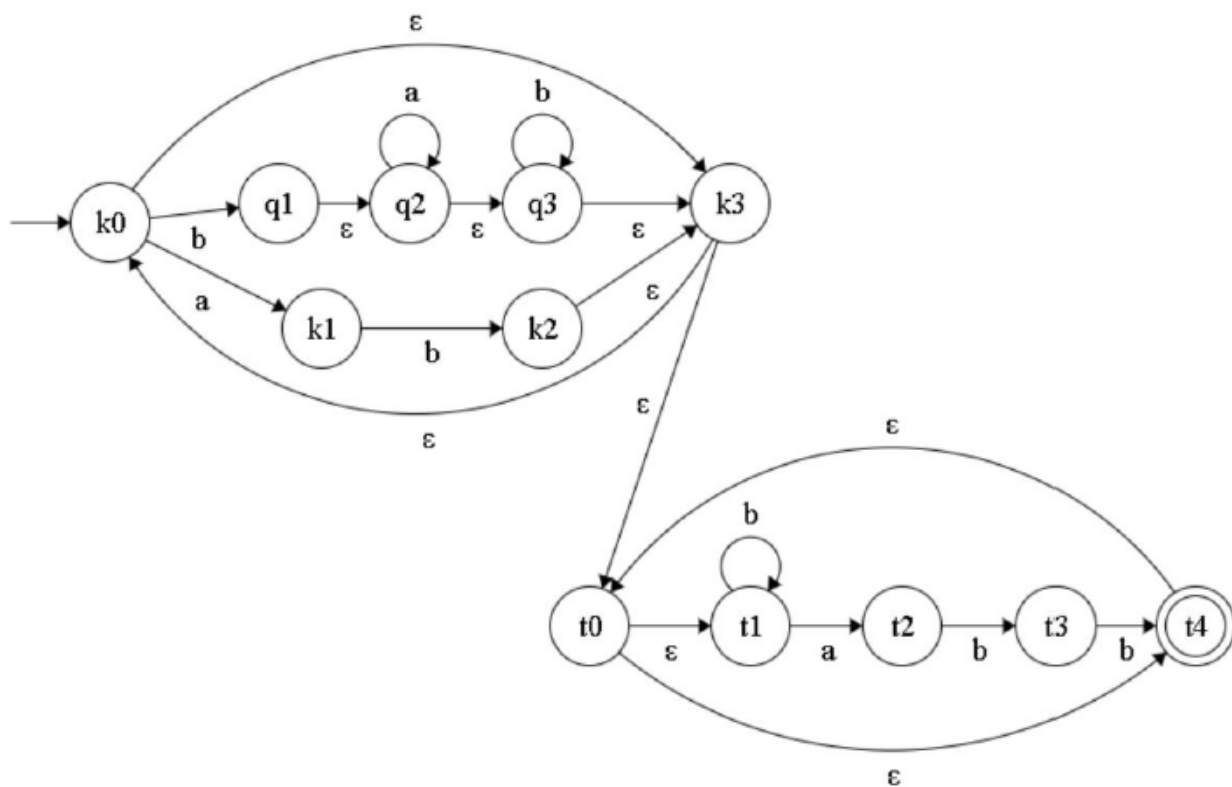
$(b \cup ab)^* :$



(b



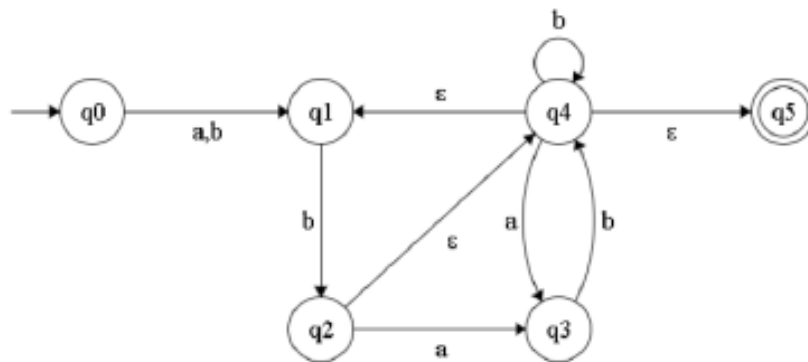
حال که برای $(b^*abb)^*$ و $(ab \cup ba^*b^*)^*$ توانستیم nfa طراحی کنیم، این 2 را کانکت می کنیم:



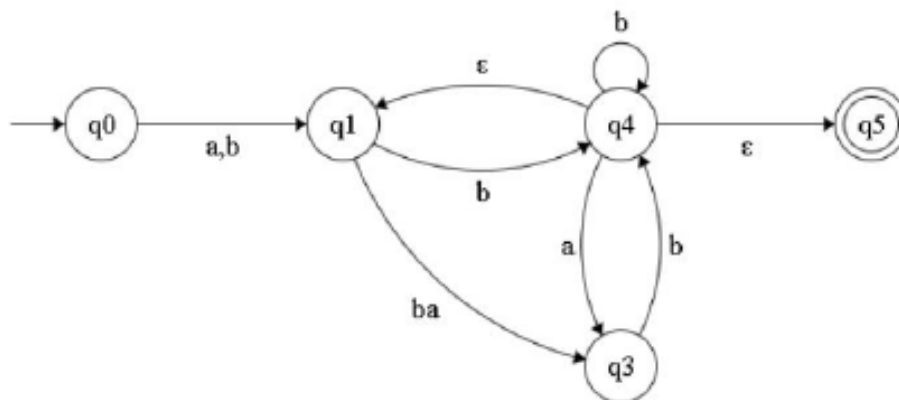
۲-۱

(الف)

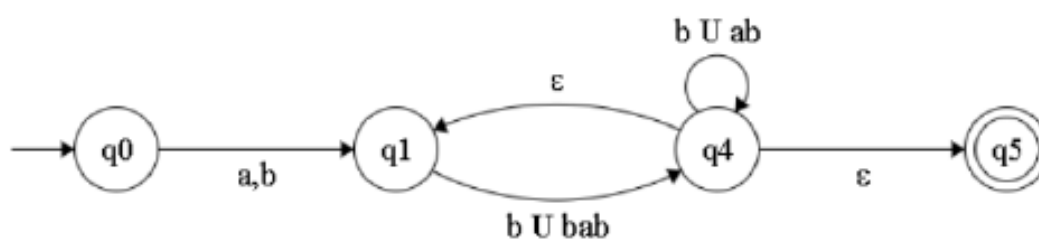
اول از همه این ماشین را به یک gnfa تبدیل می کنیم، سپس مرحله به مرحله استییت ها را حذف می کنیم:



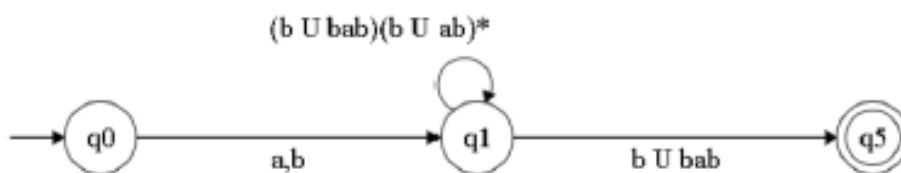
حذف استییت q2:



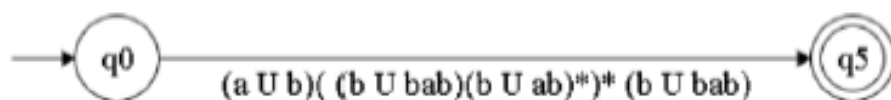
حذف استییت q3:



حذف استیت q4:

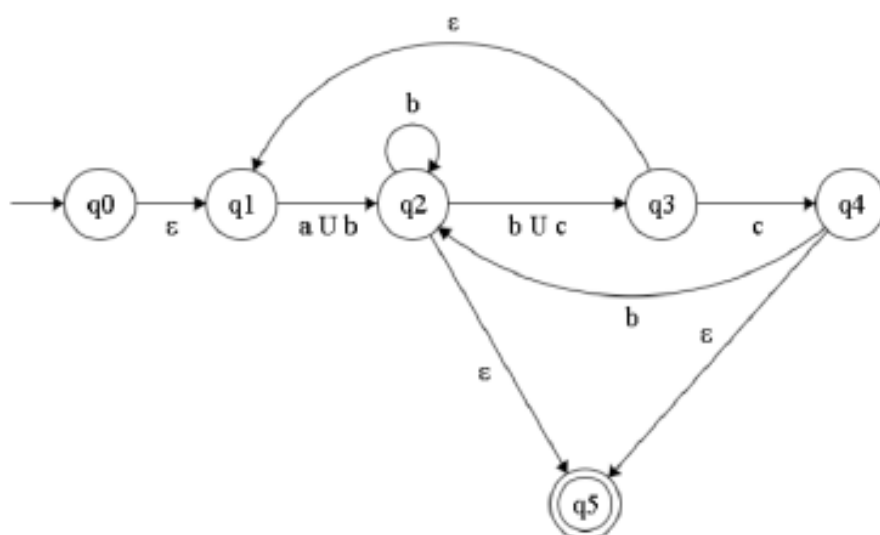


حذف استیت q1:

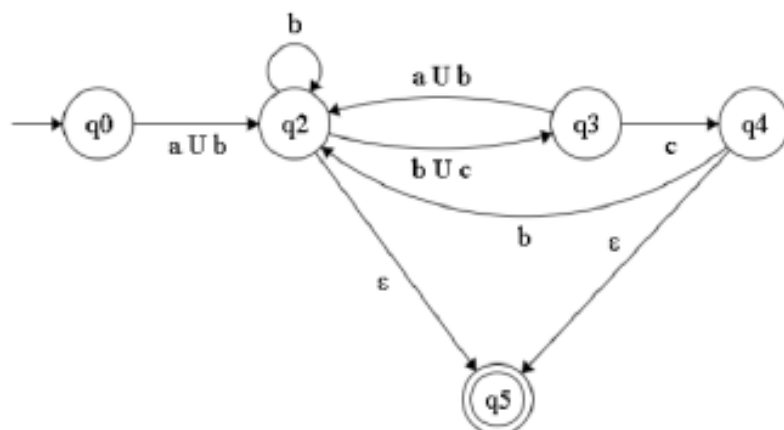


(ب)

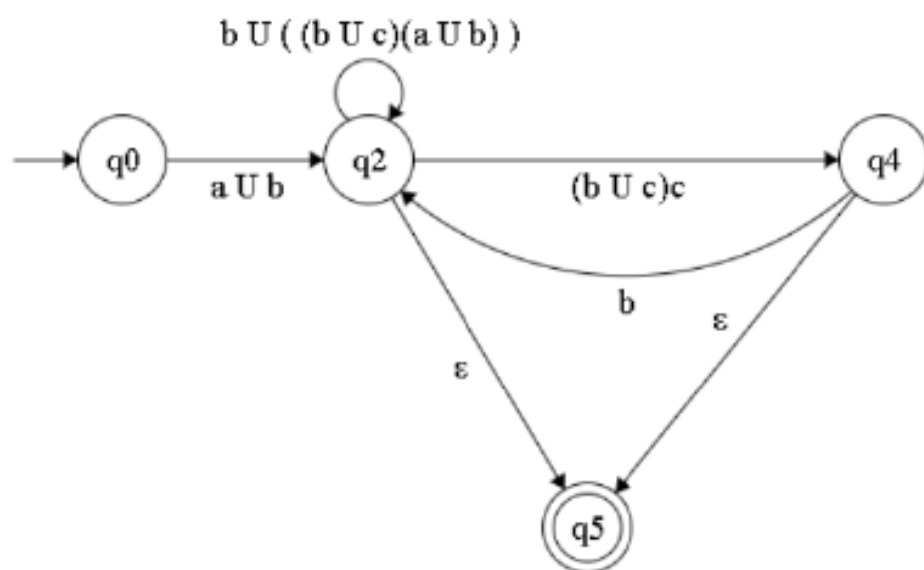
اول این ماشین را به یک gnfa تبدیل می کنیم:



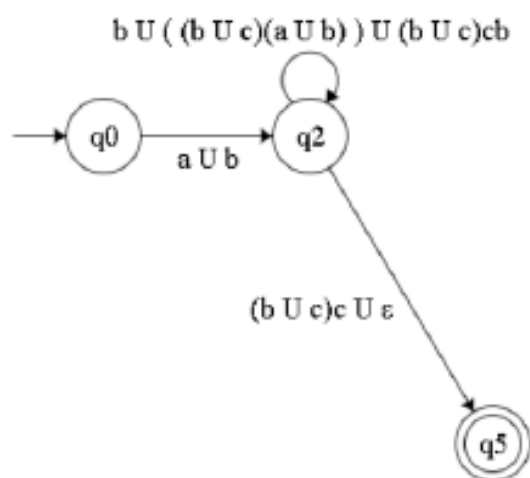
اول میاییم و q1 را حذف می کنیم:



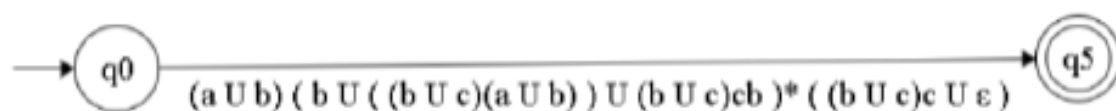
سپس q_3 را حذف می کنیم:



حذف q_4 :



و در نهایت با حذف q_2 داریم:



۲ اثبات نامنظم بودن زبان‌ها

۱-۲

(L₁)

در زبان L_1 می‌دانیم که n و m و k هر 3 برابر نبوده‌اند. پس برای اثبات نامنظم بودن این زبان تلاش می‌کنیم با کمک لم *pumping* رشته‌ای پیدا کنیم که تعداد a, b و c های برابری داشته باشد.

فرض کنید *dfa* ای که زبان L_1 را توصیف می‌کند، استیت‌هایش q_0 تا q_s باشد.

فرض کنید که یک رشته به شکل $x = a^n b^m c^m$ داریم که $m > n > s+1$ ، پس این رشته عضو L_1 است.

دقت کنید که $s+1$ حرف اول رشته x همه حرف a هستند، و اگر در *dfa* این زبان نگاه کنیم، حتما در این $s+1$ حرف اول استیتی وجود داشته که 2 بار تکرار شده، که این به ما اجازه *pump* کردن میدهد، اما نکته‌ای که می‌خواهیم به آن اشاره کنم این است که مقدار عدد m هیچ تاثیری در طول رشته‌ای که *pump* می‌کنیم ندارد، چون مقدار m هرچیزی باشد، $s+2$ استیتی که اول کار به آنها می‌رویم تغییری نمی‌کنند، چون ماشین ما یک *dfa* است و مستقل از مقدار m ، $s+2$ استیتی که در اول به آنها می‌رفتیم یکتا تایین میشوند. پس اگر طول رشته‌ای که قراره *pump* کنیم t باشد، مقدار m را برابر با $n+t$ قرار میدهیم، در این صورت با یک بار *pump* کردن می‌فهمیم که $a^{n+t} b^{n+t} c^{n+t}$ عضو زبانمان است، اما این تناقض است، و از این تناقض می‌فهمیم این زبان منظم نبوده است.

(L₂)

طبق لم *pumping* می‌دانیم که عدد k وجود دارد که برای هر رشته که طولش حداقل k است داشته باشیم که آن را می‌توان به صورت xyz نوشت که $|xy| \leq k$ و رشته y را می‌توان *pump* کرد. پس اگر اینجا p و q را طوری انتخاب کنیم که $pq \geq k$ داریم که رشتمان را می‌توانیم به صورت xyz بنویسیم به طوری که $|y| > 0$ و $xy^n z$ عضو L_2 است.

چون رشته ما کاملاً از حرف a تشکیل شده است، لم پامپینگ در اصل دارد می‌گوید که اگر $|y| = t$ پس $a^{pq+nt} \in L_2$

$$n=pq \Rightarrow a^{pq+pq} \in L_2 \Rightarrow$$

در نتیجه $pq(t+1)$ را باید بتوان به صورت ضرب 2 عدد اول نوشت، اما دقت کنید که تا همینجا به صورت ضرب 2 تا عدد اول ضرب در $(t+1)$ آن را نوشته ایم، و با توجه به یکتایی تجزیه اعداد به عوامل اول، امکان نداشته این عدد را هیچ جوره به صورت ضرب فقط 2 عدد اول بنویسیم. پس ازین تناقض می‌فهمیم که این زبان منظم نبوده است.

فرض خلف کنید و فرض کنید این زبان منظم است.

قبل از حل سوال به یک نکته ای اشاره می کنم، اگر فرض کنیم dfa این زبان ما، p تا استیث داشته باشد، و فرض کنیم z یک رشته ای باشد که زبان L آن را میپذیرد، به طوری که p تا حرف اول رشته z تماما از حرف b باشد، در این صورت اگر همان روند لم یامینگ را پیش ببریم، میبینیم که رشته ای مثل w وجود دارد که کاملاً از حرف b تشکیل شده باشد و می توانیم رشته w را در یک جایی از p تا حرف اول z ، تزریق کنیم.

حالا نکته ای که به آن می خوام دقیق تر اشاره کنم این است که طول رشته w از ادامه رشته z مستقل است، یعنی برای هر رشته ای دیگه ای در زبان L مثل z' که p حرف اول z' تماما حرف b باشد، رشته ای که در زبان b' می توانیم pump کنیم با رشته ای که در زبان z می توانیم pump کنیم برابر است، چون p حرف اول z و z' در dfa زبان L دقیقاً از استیث های یکسانی گذشته اند.

در نتیجه برای هر z در زبان L ، که p حرف اولش b باشد میتوان گفت که عدد ثابت ناصفر t وجود دارد که

$$b^{nt}zeL$$

که اینجا t همان طول رشته w است.

فرض کنید که $n \geq p$ عددی زوج باشد، x و y را به صورت زیر تعریف می کنیم:

$$x = \underbrace{bbb \dots b}_n \underbrace{ccc \dots c}_{2t-1}$$

$$y = \underbrace{ccc \dots c}_{n+2t-1}$$

دقت کنید که این t همان مقدار t ای است که قبل تر بهش اشاره کردیم.

حال داریم که:

$$\begin{aligned} \text{Value}(x) &= \underbrace{b \times b \times b \dots \times b}_n \times \underbrace{c \times c \times c \dots \times c}_{2t-1} = a \times \underbrace{b \dots \times b}_{n-2} \times \underbrace{c \times c \times c \dots \times c}_{2t-1} = \dots = \\ &= a \times \underbrace{c \times c \times c \dots \times c}_{2t-1} = \underbrace{c \times c \times c \dots \times c}_{2t-1} = a \times \underbrace{c \times c \times c \dots \times c}_{2t-3} = \dots = a \times c = c \end{aligned}$$

$$\text{Value}(y) = \underbrace{c \times c \times c \dots \times c}_{n+2t-1} = a \times \underbrace{c \times c \times c \dots \times c}_{n+2t-3} = \underbrace{c \times c \times c \dots \times c}_{n+2t-3} = \dots = a \times c = c$$

پس داریم که $|x|=|y|$ و $\text{Value}(x)=\text{Value}(y)$ در نتیجه می توان گفت $xy \in L$.

حالا دقت کنید که xy رشته ای است که p تا حرف اول آن همه b هستند، در نتیجه داریم که:

$$b^{4t}xy \in L$$

پس داریم که:

$$b^{4t}xy = \underbrace{bbb \dots b}_{4t} \underbrace{bbb \dots b}_n \underbrace{ccc \dots c}_{n+4t-2} = \underbrace{bbb \dots b}_{n+4t} \underbrace{ccc \dots c}_{n+4t-2}$$

و اگر قرار باشد رشته بالا به 2 قسمت x' و y' تقسیم شود که طول x' و y' برابر باشد، داریم که:

$$x' = \underbrace{bbb \dots b}_{n+4t-1}$$

$$y' = b \underbrace{ccc \dots c}_{n+4t-2}$$

پس داریم که $x'y' \in L$ ، پس در نتیجه Value این 2 رشته باید برابر باشد:

$$Value(x') = \underbrace{b \times b \times b \dots \times b}_{n+4t-1} = a \times \underbrace{b \dots \times b}_{n+4t-3} = \dots = a$$

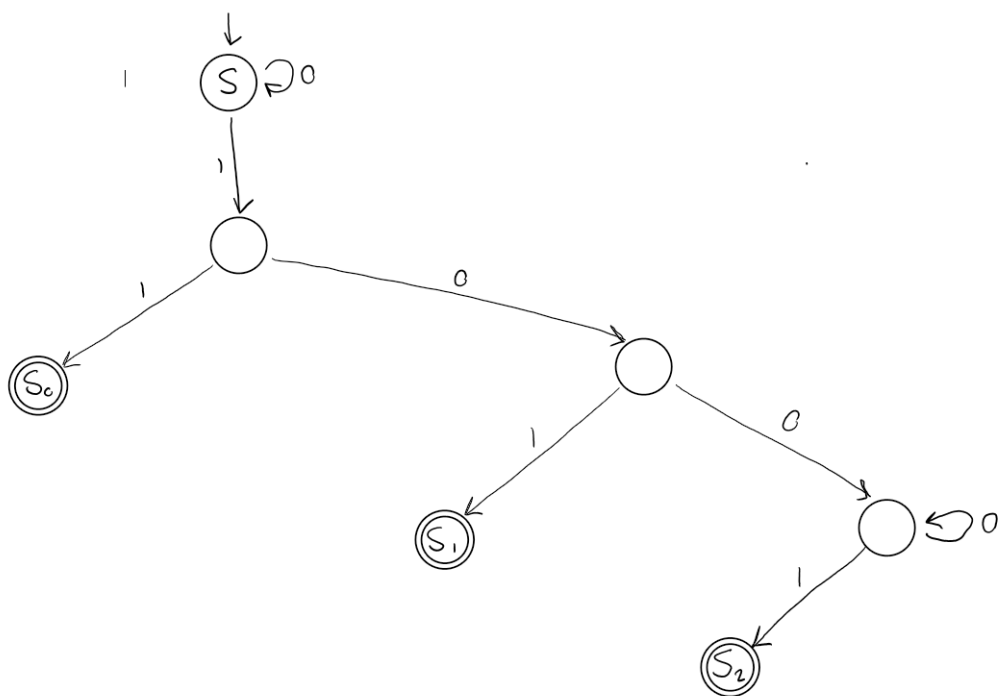
$$Value(y') = b \times \underbrace{c \times c \times c \dots \times c}_{n+4t-2} = b \times \underbrace{c \times c \times c \dots \times c}_{n+4t-3} = \dots = b \times c = b$$

که این تناقض است، و از این تناقض می فهمیم که این زبان منظم نبوده است.

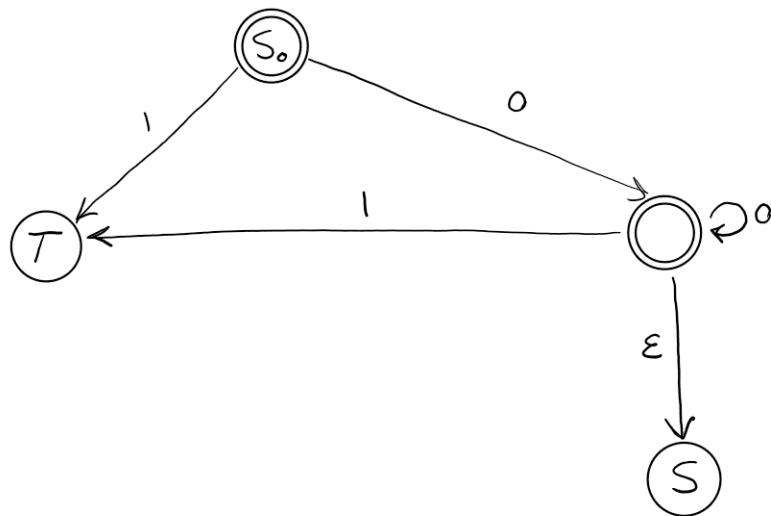
۳. ماشین‌های متناهی توسعه یافته (امتیازی)

۱.۳ ماشین متناهی دوجهته

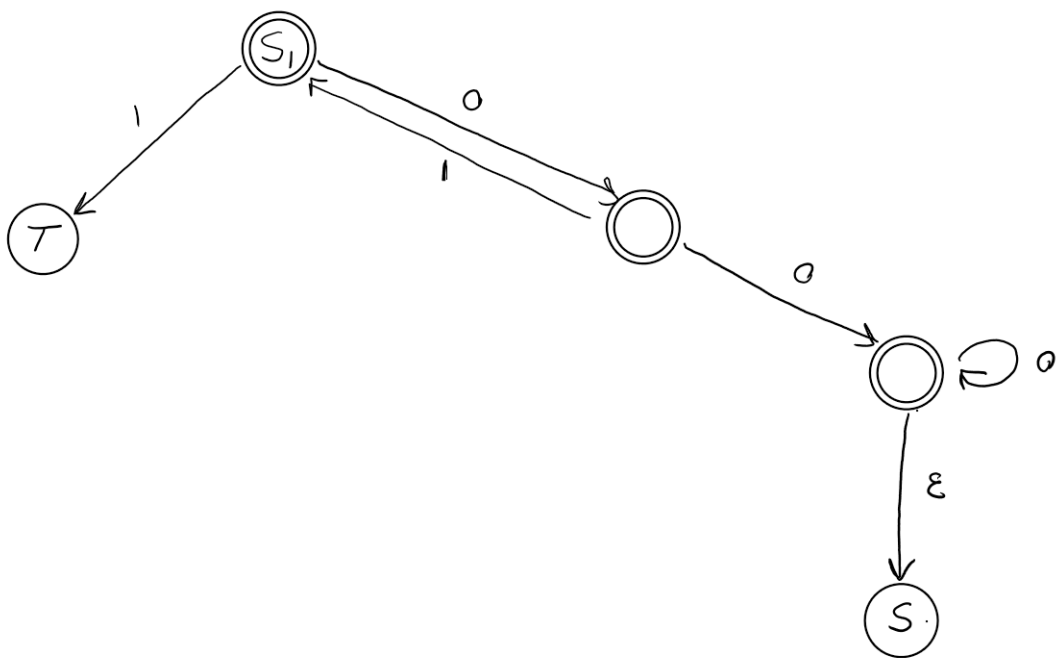
طراحی NFA ما به صورت زیر است:



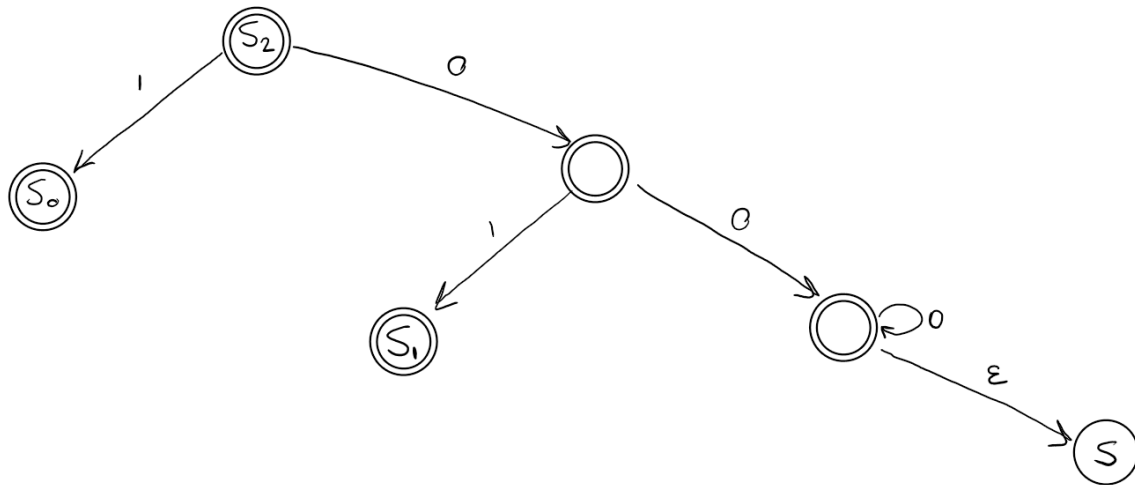
که در آن ادامه استیت S_0 به صورت زیر است:



و ادامه استیت S_1 به صورت زیر است:



و ادامه استیت S_2 به صورت زیر است:



دقت میکنیم که استیت T استیت ریجکت است و با هر ورودی از T به T میرویم. همچنین روش به دست آمدن آن از طریق زبانی که میپذیرد است و برای درک بهتر آن زبان به شکل زیر دقت میکنیم:

$0 \bar{x}_0$ | $0 \bar{x}_1$ | $0 \bar{x}_2$ | $0 \bar{x}_3$ | $0 \bar{x}_4$ | ... | $0 \bar{x}_5$...

که رشته ورودی را بر حسب ۱ های آن جداسازی میکنیم و بین هر دو ۱ متوالی تعداد ۰ های بینشان را x_i میگیریم و چند تا شرط ساده برای پذیرفتن رشته وجود دارد به این صورت که ما در ماشین هر مرتبه ۲ تا از x_i و x_{i+1} را در نظر میگیریم و بنا به حالت های مختلف این ۲ (به فرض ۰ بودن x_{i+1} یا ۱ بودن x_{i+1} و بیشتر مساوی ۲ بودن x_{i+1}) تصمیم میگیریم که آیا ادامه رشته میتواند مورد پذیرش باشد یا نه و یا اینکه استیت بعدی مان چی است را تصمیم میگیریم و اگر دقت شود استیت های S_0 برای حالتی است که $x_i = 0$ و استیت S_1 برای حالتی است که $x_i = 1$ و استیت S_2 برای حالتی است که $x_i \geq 2$ و در ادامه هر استیت حالت های مختلف برای x_{i+1} بررسی شده است. در نتیجه از درستی طراحی مان نیز مطمئن هستیم.

۲.۳ ماشین متناهی دوجهته‌ی علامت‌گذار

در ابتدا برای سادگی نمایش فرض میکنیم $\vdash$ برای شروع رشته استفاده میشود و نماد $\dashv$ برای پایان رشته.

همچنین $\triangleright$ نماد جدید تعریف میکنیم $\triangleleft$ و $\triangleright$ نماد هایی برای نشان دادن سر و ته یک عملیات در بخشی از یک رشته است.

ابتدا تابع $P : \Sigma \rightarrow \Sigma \cup \{\triangleleft, \triangleright\} \cup \Sigma^\diamond$ را به صورت زیر تعریف میکنیم:

$$P(a_1 a_2 \dots a_n) = a_1 \dots a_n \triangleleft \triangleright a_1^\diamond a_2 \dots a_n \triangleleft \triangleright a_1 a_2^\diamond \dots a_n \triangleleft \dots \triangleright a_1 a_2 \dots a_n^\diamond \triangleleft \triangleright a_1 a_2 \dots a_n$$

که در اینجا داریم که $\Sigma^\diamond$ همان مجموعه $a_i^\diamond$ ها هستند که $a_i^\diamond$ نماد خانه علامت گذاشته شده است.

اگر ماشین مان را M بنامیم ما در ابتدا یک ماشین $2DFA$ به نام M' میسازیم که معادل با M است و به دلیل اینکه میدانیم

$2DFA$ ها فقط زبان های منظم را میپذیرند آنگاه مسئله حل میشود.

حال اگر ماشین M به صورت زیر تعریف شود:

$$M = \{Q, \Sigma, \delta, q_1, F\}$$

آنگاه M' را به صورت زیر تعریف میکنیم:

$$M' = \{Q', \Sigma', \delta', q'_1, F'\}$$

به طوری که $\Sigma' = \Sigma \cup \{\triangleleft, \triangleright\} \cup \Sigma^\diamond$ و رشته ω در M پذیرفته است اگر و تنها اگر $P(\omega)$ در M' پذیرفته باشد.

ماشین M' به اینگونه کار میکند که هر گاه خانه s علامت زده شود، آنگاه ماشین M' در بازه s خود که همان زیر رشته $\vdash$

$\dashv$ $a_1 a_2 \dots a_s^\diamond \dots a_n$ در $P(\omega)$ است فعالیت میکند و تا زمانی که خانه s علامت زده بماند، ماشین M' نیز در همین زیر رشته

میمانند.

همچنین در هر لحظه اگر خانه مثل $\{ \vdash, \dashv \} \cup \Sigma$ دارای علامت باشد آن را با $a^\bullet$ نشان میدهیم.

حال کافیسست تابع δ' را تعریف کنیم:

$$\delta'(q, a) = \begin{cases} \text{if } \delta(q, a, N) = (q', d, U) \text{ for some } q, q' \in Q, a \in \Sigma, d \in \{L, R\} : \delta'(q, a) = (q', d) \\ \text{if } \delta(q, \vdash, N) = (q', d, U) \text{ for some } q, q' \in Q, d \in \{R\} : \delta'(q, \triangleright) = (q', d) \\ \text{if } \delta(q, \dashv, N) = (q', d, U) \text{ for some } q, q' \in Q, d \in \{L\} : \delta'(q, \triangleleft) = (q', d) \\ \text{if } \delta(q, a^\bullet, Y) = (q', d, U) \text{ for some } q, q' \in Q, a \in \Sigma, d \in \{L, R\} : \delta'(q, a^\diamond) = (q', d) \\ \text{if } \delta(q, \vdash^\bullet, Y) = (q', d, U) \text{ for some } q, q' \in Q, d \in \{R\} : \delta'(q, \vdash) = (q', d) \\ \text{if } \delta(q, \dashv^\bullet, Y) = (q', d, U) \text{ for some } q, q' \in Q, d \in \{L\} : \delta'(q, \dashv) = (q', d) \end{cases}$$

برای سادگی توضیحات فرض شده است که نماد $L^\bullet$ برای این است که علاوه بر حرکت اشاره گر به چپ است بلکه علامت ما نیز به خانه چپ حرکت میکند و $R^\bullet$ نیز به همین صورت تعریف میشود.

همچنین برای سادگی فرض شده است در M' میتوان نه به چپ و نه به راست رفت در یک مرحله و برای آن به جاب L و R از $NULL$ استفاده میشود.

دقت میکنیم که اگر ما در یک لحظه خانه s علامتدار باشد و این علامت حذف شود و به خانه راست آن برود، در ماشین M' که تا قبل از این جابه‌جایی طبق توضیحات در زیر رشته s بوده حال باید به زیر رشته $s + 1$ برود و در آن زیر رشته به دنبال خانه علامت خورده جدید باشد و پس از پیدا کردن آن به روند عادی خود ادامه می‌دهد.

پس برای حرکت به راست داریم:

$$\delta'(q, a) = \left\{ \begin{array}{l} \text{if } \delta(q, a^\bullet, Y) = (q', R^\bullet, M) \text{ for some } q, q' \in Q, a \in \Sigma, : \left\{ \begin{array}{l} q'_R \text{ is a passing state and is} \\ \text{a copy of } q' \\ \delta'(q, a^\diamond) = (q'_R, R) \\ \delta'(q'_R, a) = (q'_R, R) \\ \delta'(q'_R, \triangleleft) = (q'_R, R) \\ \delta'(q'_R, \triangleright) = (q'_R, R) \\ \delta'(q'_R, a^\diamond) = (q', NULL) \\ \delta'(q'_R, \neg) = (q', NULL) \end{array} \right. \\ \\ \text{if } \delta(q, \vdash^\bullet, Y) = (q', R^\bullet, M) \text{ for some } q, q' \in Q, : \left\{ \begin{array}{l} q'_R \text{ is a passing state and is} \\ \text{a copy of } q' \\ \delta'(q, \vdash) = (q'_R, R) \\ \delta'(q'_R, a) = (q'_R, R) \\ \delta'(q'_R, \triangleleft) = (q'_R, R) \\ \delta'(q'_R, \triangleright) = (q'_R, R) \\ \delta'(q'_R, a^\diamond) = (q', NULL) \\ \delta'(q'_R, \neg) = (q', NULL) \end{array} \right. \end{array} \right.$$

همچنین داریم که اگر ما در یک لحظه خانه s علامتدار باشد و این علامت حذف شود و به خانه چپ آن برود، در ماشین M' که تا قبل از این جابه‌جایی طبق توضیحات در زیر رشته s بوده حال باید به زیر رشته $s - 1$ برود و در آن زیر رشته به دنبال خانه علامت خورده جدید باشد و پس از پیدا کردن آن به روند عادی خود ادامه می‌دهد.

پس برای حرکت به چپ داریم:

$$\delta'(q, a) = \left\{ \begin{array}{l} \text{if } \delta(q, a^\bullet, Y) = (q', L^\bullet, M) \text{ for some } q, q' \in Q, a \in \Sigma, : \\ \left\{ \begin{array}{l} q'_L \text{ is a passing state and is} \\ \text{a copy of } q' \\ \delta'(q, a^\diamond) = (q'_L, L) \\ \delta'(q'_L, a) = (q'_L, L) \\ \delta'(q'_L, \triangleleft) = (q'_L, L) \\ \delta'(q'_L, \triangleright) = (q'_L, L) \\ \delta'(q'_L, a^\diamond) = (q', NULL) \\ \delta'(q'_L, \neg) = (q', NULL) \end{array} \right. \\ \\ \text{if } \delta(q, \neg^\bullet, Y) = (q', L^\bullet, M) \text{ for some } q, q' \in Q, : \\ \left\{ \begin{array}{l} q'_L \text{ is a passing state and is} \\ \text{a copy of } q' \\ \delta'(q, \neg) = (q'_L, L) \\ \delta'(q'_L, a) = (q'_L, L) \\ \delta'(q'_L, \triangleleft) = (q'_L, L) \\ \delta'(q'_L, \triangleright) = (q'_L, L) \\ \delta'(q'_L, a^\diamond) = (q', NULL) \\ \delta'(q'_L, \neg) = (q', NULL) \end{array} \right. \end{array} \right.$$

پس طبق توضیحات بالا میتوان گفت که $Q' = Q \cup Q_L \cup Q_R$ که Q_L و Q_R شامل q'_L و q'_R ها هستند، همچنین داریم که $q'_1 = q_1$ و $F' = F$.

حال تعریف ماشین $2DFA$ ما که همان M' است کامل است و کافیت درستی آن را نیز بیان کنیم.

برای اثبات درستی لم زیر را ابتدا اثبات میکنیم:

لم ۱. در ماشین M با ورود رشته w میتوان به استیت q برسد به طوری که اشاره گر به خانه h اشاره میکند و خانه p علامت زده باشد،

اگر و تنها اگر با ورودی رشته $P(w)$ به ماشین M' به استیت q برسد و اشاره گر به خانه h از زیر رشته p در وردی، اشاره کند.

اثبات:

برای سادگی صرفا کلیت اثبات آمده است.

برای بخش اول اثبات کفایت استقرا بزنیم روی تعداد مراحل حرکت در M و برای بخش دیگر آن نیز استقرا میزنیم روی تعداد دفعات

که ماشین M' روی یکی از استیت های Q قرار دارد. (طبق تعریفی که از P شده و تابع δ اثبات بدیهی است.)

حال طبق لم داریم که اگر در M با رشته ω با شروع از استیت $q_1 = q'_1$ به یک استیت $q \in F = F'$ برسد آنگاه در ماشین M' با

شروع از استیت $q_1 = q'_1$ با رشته $P(\omega)$ به یک استیت $q \in F = F'$ میرسد، همچنین اگر در M' با شروع از $q_1 = q'_1$ با رشته

$P(\omega)$ به یک استیت $q_1 = q'_1, q \in F = F'$ برسیم به دلیل اینکه طبق تعریف q یک passing through state نمیباشد

آنگاه در ماشین M با رشته ω با شروع از استیت $q_1 = q'_1$ به یک استیت $q \in F = F'$ میرسیم، در نتیجه این ۲ ماشین معادل هم

هستند و مسئله حل است.